Module 1 - Python Fundamentals

1.1 - Introduction to Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and powerful capabilities. Created by **Guido van Rossum** and first released in 1991, Python has grown into one of the most popular languages for automation, web development, data science, and cybersecurity in especially in offensive and defensive security.

Python Philosophy

Python follows the **Zen of Python**, a set of design principles. You can see them by running:

```
import this
```

Key ideas:

- **Simple is better than complex**
- **Readability counts**
- **There should be one — and preferably only one — obvious way to do it**

Python Versions (Python 2 vs Python 3)

> This course uses **Python 3**, which is now the official and maintained version.

- **Python 2** is outdated and no longer supported (EOL in 2020)
- **Python 3.x** is faster, more secure, and includes features such as `print()` as a function and better Unicode support

Installing Python

🪟 On Windows:

- Download from https://www.python.org/downloads/
- **Important**: Check the box "Add Python to PATH" during installation

On Linux:

```
sudo apt update
sudo apt install python3 python3-pip
```

On macOS (Homebrew):

```
brew install python3
```

Running Python Code

Option 1: Interactive mode (REPL)

Open terminal and type:

```
python3
```

You'll get:

```
>>> print("Hello, world!")
Hello, world!
```

Option 2: Running `.py` scripts

Create a file:

```
nano hello.py
```

Inside:

```
print("Hello from a script!")
```

Run:

```
python3 hello.py
```

Setting Up an IDE

Recommended environments for coding:

- **VS Code** (with Python extension)
- **PyCharm** (Community Edition is free)
- **Jupyter Notebooks** (for interactive scripts and demos)
- **Thonny** (great for beginners)

Practice Task

Create a simple script named `intro.py` with the following code:

```
# intro.py
print("Welcome to Offensive Python!")
print("This is your first Python script.")
```

Then run:

```
python3 intro.py
```

1.2 - Basic Data Types (Deep Dive)

Python's built-in scalar and text types hide a great deal of subtle behaviour that becomes important when you start manipulating raw network data, generating shellcode, or processing large log files. Mastering those details early saves time later when your scripts misbehave under stress.

1.2.1 Numeric Types

Type	Precision	Mutable?	Notes
int	Arbitrary	Immutable	No fixed bit-width; grows to accommodate any integer.
float	64-bit IEEE-754	Immutable	Binary floating point; rounding error is common.
complex	Two 64-bit floats	Immutable	3+4j form; handy for FFTs but rarely used in security code.
bool	Subclass of int	Immutable	True == 1, False == 0.

Integers (`int`)

```
n = 2_147_483_648          # underscores improve readability
print(n.bit_length())      # bits needed: 31
print(hex(n))              # '0x80000000'
print(n.to_bytes(4, 'big'))  # b'\x80\x00\x00\x00'
```

Key points

1. **Unlimited size** means cryptographic math (RSA, ECC) can be done without third-party big-int libraries.
2. Converting to/from bytes is essential when you craft exploit payloads or network packets.

Floats (`float`)

```
>>> 0.1 + 0.2
0.30000000000000004
```

Why it matters: timing attacks and packet timestamping often rely on precise arithmetic. Use the `decimal` module when you need exact base-10 decimals, or compare floats within tolerances:

```
from math import isclose
isclose(0.1 + 0.2, 0.3, rel_tol=1e-09)
```

Alternative exact numeric types

```
from decimal import Decimal    # fixed-point, user-defined precision
from fractions import Fraction # rational numbers
```

These appear in cryptanalysis and forensic accounting scripts where rounding is unacceptable.

1.2.2 Text Data: `str`

Unicode everywhere
Since Python 3, `str` stores Unicode code points, not raw bytes. When you receive network data:

```
raw = sock.recv(1024)        # bytes
text = raw.decode('utf-8', errors='replace')
```

Immutability
Concatenating many small strings with `+` creates intermediate objects. Prefer `"".join(parts)` or use `io.StringIO` for large loops.

Indexing, slicing

```
s = "pwned"
s[0]      # 'p'
s[-1]     # 'd'
s[1:4]    # 'wne'
```

Common operations

```
s.startswith("pw")
s.find("ne")
s.replace("pwn", "own")
```

String Formatting

Style	Example	Remarks
f-strings	f"{ip}:{port}"	Fast, readable, type-safe

Style	Example	Remarks
str.format	"{}:{}"	Good for dynamic field widths
percent	"%s:%d"	Historic; still seen in CTF code

1.2.3 Byte-Oriented Types

Type	Mutable?	Purpose
bytes	No	Immutable sequence of integers 0-255.
bytearray	Yes	In-place edits (e.g. patching shellcode).
memoryview	N/A	Zero-copy slice into another bytes-like object.

```
payload  = b"\x90" * 16 + b"\xcc"
mutable  = bytearray(payload)
mutable[0] = 0xcc                # overwrite first NOP
view = memoryview(mutable)[1:5]   # cheap slice
```

Why you care: assembling an exploit often demands byte-perfect control without allocating fresh objects.

1.2.4 Boolean Context and Truthiness

Objects evaluate to `False` when:

- numeric value is `0`
- container length is `0`

- they are the singleton `None`
- their class defines `__bool__` returning `False` or `__len__` returning `0`

```
if connection:         # relies on __bool__ method or length
    handle(connection)
```

Implementing `__bool__` in custom classes lets you write expressive guard clauses later in your tooling.

1.2.5 `None`

Singleton sentinel used to signal “no value”, not “False”. Identity (`is`) comparisons are the idiom:

```
if result is None:
    raise ValueError("Result missing")
```

Pitfall: `==` may be overloaded by user types; always use `is` .

1.2.6 Type Checking and Conversion

Dynamic typing means variables carry values, not types, but you can:

```
isinstance(x, (bytes, bytearray))
```

Use this sparingly; prefer *duck typing* plus try/except (EAFP) to keep code flexible.

Casting rules are explicit – Python will not silently convert `str` ↔ `int` .

1.2.7 Mutability and Identity

Immutable objects are safe as dictionary keys and hashable; mutable ones are not.

```
seen = {b'\x90\x90': "NOP sled"}   # bytes OK
cache = {(1, 2): "coords"}         # tuple OK

key = bytearray(b'abc')
# seen[key]  # TypeError: unhashable
```

Identity (`is`) checks memory address; equality (`==`) checks value. Two small integers or short strings may point to the same interned object, but do not rely on it.

1.2.8 Performance Considerations

- CPython uses reference counting; each object has a small overhead (28 bytes on 64-bit builds). Many tiny integers or strings can inflate memory usage in long-running agents.
- Use `array.array` , `struct.pack` , or third-party `numpy` when you need dense numeric storage.

1.2.9 Security-Relevant Examples

Parsing a Hex-Encoded Shellcode String

```
hexstr = "fc4883e4f0e8"
shellcode = bytes.fromhex(hexstr)
```

Building a DNS Query Packet

```
import struct
query_id = 0x1337
flags = 0x0100
qcount = 1
header = struct.pack("!HHHHHH", query_id, flags, qcount, 0, 0, 0)
```

1.2.10 Exercises

1. **Endianness**
   Write a function `int_to_little_endian(n, length)` that converts any positive integer to a little-endian byte sequence of fixed length.
2. **Safe Float Comparison**
   Implement `def almost_equal(a, b, rel_tol=1e-9, abs_tol=0.0)` reproducing `math.isclose` without using the module.
3. **Mutable XOR**
   Create a function `xor_bytes(data: bytearray, key: int)` that XORs each byte with a one-byte key in place.
4. **Truthiness Audit**
   Write a class `OpenPort` with attributes `port` and `status`, and define `__bool__` so that the instance is truthy only when `status == "open"`.
5. **String Builder Benchmark**
   Compare execution time between concatenating 10 000 lines with `+=` versus collecting them in a list and `"".join`.
   Use `time.perf_counter`.

1.2.1 - Basic Data Types (Codes)

1.2.2 – Integers and Floats

Python supports both whole numbers (**integers**) and decimal numbers (**floats**).

```python
# Integer
x = 10

# Float
y = 3.14

# Arithmetic operations
print(x + y)    # 13.14
print(x * 2)    # 20
print(y / 2)    # 1.57
print(x // 3)   # Floor division -> 3
print(x % 3)    # Modulo -> 1
print(2 ** 4)   # Exponentiation -> 16
```

1.2.3 – Strings

Strings are sequences of characters. They can be defined with single, double, or triple quotes.

```python
name = 'Alice'
message = "Hello, " + name
print(message)

# Multiline string
bio = """This is a
multiline string."""
```

String operations

```python
print(name.upper())      # 'ALICE'
print(name.lower())      # 'alice'
print(name[0])           # 'A'
```

```
print(name[-1])          # 'e'
print(name[1:4])         # 'lic'
print(len(name))         # 5
```

String formatting

```
age = 30
print(f"{name} is {age} years old.")     # Using f-strings
print("{} is {} years old.".format(name, age))  # Using format()
```

1.2.4 – Booleans

Booleans are used for logic and control flow.

```
is_active = True
is_admin = False

print(is_active and is_admin)   # False
print(is_active or is_admin)    # True
print(not is_active)            # False
```

Comparison operators

```
x = 5
y = 10

print(x == y)   # False
print(x != y)   # True
print(x < y)    # True
print(x >= 5)   # True
```

1.2.5 – Type Checking and Conversion

You can check and convert data types as needed.

```
print(type(42))           # <class 'int'>
print(type("hello"))      # <class 'str'>
print(type(3.14))         # <class 'float'>

# Type conversion
print(int(3.99))          # 3
print(float(2))           # 2.0
print(str(42))            # '42'
print(bool(""))           # False
```

1.2.6 – None

`None` is a special constant that represents the absence of a value.

```
x = None
print(x is None)   # True
```

1.3 - Core Data Structures

Python's built-in collection types— `list` , `tuple` , `dict` , and `set` —form the backbone of most security automation. Choosing the right structure affects both speed (critical in network sweeps) and correctness (deduplication, order preservation, constant-time look-ups). Below is a practical deep dive aimed at offensive-security scripting.

1.3.1 Lists (`list`)

Characteristics

Aspect	Details
Mutability	Yes (in-place edits, append, pop, etc.)
Ordering	Preserves insertion order
Storage	Dynamic array of object pointers; O(1) random access
Hashable	No

```
ports = [21, 22, 80, 443]
ports.append(8080)
ports[0] = 2121
del ports[1]             # remove port 22
```

Slicing and Stride

```
top3 = ports[:3]         # copy first three
rev  = ports[::-1]       # reverse without `reversed()`
```

Slicing returns a **shallow copy**; nested objects are *not* duplicated.

List Comprehensions

```
open_ports = [p for p in ports if is_open(p)]
```

Comprehensions are compiled into C-level bytecode loops—faster and clearer than manual `for` with `append`.

List as Stack / Queue

```
stack.append(item); item = stack.pop()          # LIFO
from collections import deque
queue = deque(); queue.append(x); x = queue.popleft()  # FIFO, O(1)
```

`deque` outperforms list for frequent pops from the left.

Memory Cost

Each list slot stores a pointer (8 bytes on 64-bit CPython) plus overall header—important when holding millions of scan results. If memory is tight, use `array.array`, `numpy`, or a generator pipeline.

1.3.2 Tuples (`tuple`)

Characteristics

Aspect	Details
Mutability	No
Ordering	Yes
Hashable	Yes if all elements are hashable
Storage	Fixed-size, slightly smaller than list

```
coords = (10.0, 20.5)
ip_port = ("10.0.0.5", 22)
```

Use cases

- Keys in a dictionary or set

- Return multiple values without a custom class
- Protect read-only constant sequences (e.g., default headers)

Unpacking Tricks

```
host, port = ip_port
a, *middle, z = range(10)      # starred unpacking
```

`namedtuple` and `dataclass`

```
from collections import namedtuple
Result = namedtuple("Result", "ip port status")
r = Result("10.0.0.5", 80, "open")
```

Gives attribute access while staying memory-light and immutable.

1.3.3 Dictionaries (`dict`)

Characteristics

Aspect	Details
Mutability	Yes
Ordering	Insertion order preserved (since 3.7 guaranteed)
Average look-up	O(1) via open-addressed hash table
Hashable	No

```
services = {"ssh": 22, "http": 80, "https": 443}
services["dns"] = 53
port = services.get("ftp", 21)   # default value
```

Iteration Views

```
for key, value in services.items():
    ...
```

`items()`, `keys()`, and `values()` are dynamic *views*; they reflect mutations live without copying.

Security Note: Hash-Flood Attacks

Malicious user input of many colliding keys can degrade look-ups to `O(n)`. From CPython 3.3 onward, a random hash seed mitigates this; still relevant for protocol fuzzing.

Variants

- `collections.defaultdict(list)`: auto-creates list values—ideal for grouping hosts by ASN.
- `collections.Counter`: frequency counts (e.g., top user-agent strings).
- `collections.OrderedDict`: now redundant for order but still exposes `move_to_end`.

1.3.4 Sets (`set`, `frozenset`)

Characteristics

Aspect	Details
Mutability	set yes, frozenset no

Aspect	Details
Ordering	Unordered (hash iteration order)
Membership	O(1) average
Hashable	only frozenset

```
seen_ips = {"10.0.0.5", "10.0.0.7"}
seen_ips.add("10.0.0.8")
duplicate = "10.0.0.5" in seen_ips     # True
```

Sets excel at deduplication and fast membership tests—crucial when pruning large wordlists or tracking visited nodes in graph traversal.

Mathematical Operations

```
internal = {"10.0.0.5", "10.0.0.8"}
external = {"8.8.8.8", "10.0.0.8"}
intersection = internal & external     # {'10.0.0.8'}
difference   = external - internal     # {'8.8.8.8'}
```

`frozenset` can be dictionary keys or elements of another set, enabling higher-order membership graphs.

1.3.5 Shallow vs Deep Copies

```
import copy
orig = [[1, 2], [3, 4]]
shallow = orig.copy()
deep    = copy.deepcopy(orig)
```

```
orig[0][0] = 99
# shallow[0][0] is now 99 as well; deep[0][0] remains 1
```

When caching parsed JSON or large nested structures, choose the copy strategy deliberately.

1.3.6 Choosing the Right Structure

Requirement	Recommended Type	Rationale
Preserve order, random access, frequent appends	list	contiguous memory, fast append
Immutable sequence, hashable	tuple	can be dict key
Key–value mapping with constant-time look-up	dict	built-in hash table
Fast membership test, uniqueness	set	no duplicates, O(1) in
Double-ended queue	collections.deque	O(1) append/pop both ends
Fixed-width numeric buffer	array.array	compact C-type storage

---

1.3.7 Security-Centric Examples

Deduplicating Subdomains

```
with open("subs.txt") as f:
    uniques = set(line.strip() for line in f)
for sub in sorted(uniques):
    print(sub)
```

Mapping Exploit IDs to CVE Data

```
from collections import defaultdict
cve_map = defaultdict(list)
for exploit in exploits:
    cve_map[exploit.cve_id].append(exploit)
```

Port Scan Result Cache

```
cache: dict[tuple[str, int], str] = {}
cache[("10.0.0.5", 22)] = "open"
```

1.3.8 Exercises

1. **Port Collation**
   Read a file of `ip:port` pairs, output a dict mapping each `ip` to a *sorted* list of unique ports.
2. **Set Difference Performance**
   Benchmark (`time.perf_counter`) deduplication of two million random URLs using a `set` versus list membership checks.
3. **Immutable Config Key**
   Build a `frozenset` of `(host, port, protocol)` triples and use it as keys in a dict that stores timeout values.
4. **Defaultdict Graph**
   Implement an undirected graph of internal hosts where each node's adjacency list is maintained by `defaultdict(set)` .
   Write `add_edge(a, b)` and `has_path(a, b)` using BFS.
5. **Deep Copy Pitfall**
   Show how shallow copying a list of `bytearray` payloads leads to unwanted mutation, then correct it using `copy.deepcopy` .

1.4 - Control Flow (Deep Dive)

1.4.1 Boolean Contexts

Any object can be tested for truth:

- `False` values: `False`, `None`, numeric zero, empty sequences/containers, and objects whose class defines `__bool__` returning `False` or `__len__` returning `0`.
- All other objects are `True`.

```
if items:          # True if list is non-empty
    process(items)
```

1.4.2 `if / elif / else`

```
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
else:
    grade = "C or below"
```

- **Indentation must be consistent**; typical style is 4 spaces.
- You may chain comparisons: `if 0 < x < 1:` (reads “x is between 0 and 1”).
- Complex conditions can be wrapped in parentheses to avoid backslashes:

```
if (
    request.method == "POST"
    and user.is_authenticated
    and not user.is_admin
```

```
):
    ...
```

1.4.2.1 Conditional Expression (Ternary)

```
status = "open" if code == 200 else "closed"
```

It is an expression (returns a value) and should be kept short to preserve readability.

1.4.3 `while` Loop

The `while` loop repeats **until** its condition becomes false (or a `break` occurs).

```
password = input("> ")
while password != secret:
    print("Wrong, try again")
    password = input("> ")
```

- If the condition is always true, you create an **infinite loop**—use sparingly and always with `break`.
- The loop's condition is evaluated **before** each iteration.

`while … else …`

The `else` block executes only if the loop terminates *without* a `break`.

```
n = 19
i = 2
```

```
while i * i <= n:
    if n % i == 0:
        print("Composite")
        break
    i += 1
else:
    print("Prime")          # runs only when no divisor was found
```

1.4.4 `for` Loop

Python's `for` iterates over **any** iterable object, not just numeric ranges.

```
for char in "python":
    print(char)
```

Common helpers

```
for i in range(10):                 # 0..9
    ...

for index, value in enumerate(seq):  # index + item
    ...

for a, b in zip(list1, list2):       # pairs elements
    ...
```

- `range(start, stop[, step])` is *lazy*; it produces numbers on demand.
- Unpacking works inside the loop header: `for key, val in dict.items(): …`.

`for … else …`

Same semantics as `while … else …`; rarely used but valuable for search-oriented loops.

```
for user in users:
    if user.id == target:
        found = user
        break
else:
    found = None   # only reached if loop did not break
```

1.4.5 Loop Control Statements

Statement	Effect
break	Exit the **innermost** loop immediately.
continue	Skip the remainder of the current iteration and start the next.
pass	Do nothing (placeholder for an empty block).

```
for item in items:
    if item in skip_set:
        continue          # ignore this one
    if item == sentinel:
        break             # stop looping
    process(item)
```

1.4.6 Comprehensions

Comprehensions build new collections concisely.

```
squares      = [n*n for n in range(10)]                    # list
unique       = {word.lower() for word in words}            # set
inverted     = {v: k for k, v in table.items()}            # dict
gen          = (f(x) for x in data)                        # generator
```

- A comprehension **has its own local scope** (since Python 3).
- Keep them readable—avoid deeply nested comprehensions; refactor into named functions when needed.

1.4.7 Assignment Expression (`:=` , “walrus”)

Introduced in Python 3.8, allows assignment inside an expression.

```
while (line := file.readline()):
    print(line.strip())
```

Use with restraint; it can improve clarity in some `while`-loops but harm it elsewhere.

1.4.8 Structural Pattern Matching (`match … case` , 3.10+)

Pattern matching is a *multi-way* branch that can destructure objects.

```
match command.split():
    case ["GET", path]:
```

```
        handle_get(path)
    case ["POST", path, data]:
        handle_post(path, data)
    case _:
        print("Unknown command")
```

- Patterns can include literals, variable capture, wildcards (`_`), guards (`if condition`), and type checks.
- The first matching `case` runs; no fall-through.

1.4.9 Scope Rules in Loops

- A loop **does not** create a new scope; variables assigned inside are visible after the loop.
- **Comprehensions** *do* create an internal scope—loop variables are not leaked:

```
nums = [1, 2, 3]
squares = [n*n for n in nums]
print('n' in globals())     # False
```

1.4.10 Idiomatic Guidelines

1. Prefer `for item in iterable` to numeric indexing (`range(len(seq))`) unless you need indices.
2. In a search loop, combine `for … else …` or encapsulate logic in a helper function to avoid flag variables.
3. Replace flag variables with early `return` inside functions for clarity.
4. Keep comprehensions single-layer; nested logic often reads better as explicit loops.
5. Use `enumerate` instead of manual counters.
6. Choose `itertools` utilities (`cycle` , `islice` , `product` , etc.) before writing nested loops from scratch.

1.4.11 Practice Exercises

1. **FizzBuzz with Assignment Expression**
   Write a function `fizzbuzz(n)` that prints numbers 1…n, substituting “Fizz”, “Buzz”, or “FizzBuzz” using a single `for` loop.
2. **Find First Duplicate**
   Implement `first_duplicate(seq)` that returns the first value that appears twice, using a `for` / `else` construct.
3. **Sentinel File Reader**
   Read lines from a file until a line containing only `STOP\n` is found; count how many non-empty lines were read.
4. **Cartesian Product Comprehension**
   Given two lists of numbers, create a list of `(a, b, a*b)` tuples for every combination where `a*b` is even.
5. **Pattern-Match Calculator**
   Accept strings like `"ADD 4 5"` or `"MUL 3 9"`. Use `match … case` to parse and compute the result.

1.5 - Functions

Functions are Python’s primary unit of code reuse and abstraction.
They are **first-class objects**: you can assign them to variables, store them in containers, pass them as arguments, and return them from other functions.

1.5.1 Defining a Function

```
def greet(name):
    """Return a friendly greeting."""
    return f"Hello, {name}"
```

- `def` creates a function object and binds it to the identifier `greet`.
- The **docstring** is the first string literal in the body; tools such as `help()` and IDEs display it.

- The `return` statement ends execution and sends a value back to the caller. If no `return` is executed, the function implicitly returns `None`.

1.5.2 Argument-Passing Semantics

Python uses **"pass-by-object-reference."**
A parameter receives a reference to the original object; mutating that object inside the function mutates it for the caller.

```
def append_x(lst):
    lst.append("x")

data = ["a"]
append_x(data)
# data is now ['a', 'x']
```

Rebinding the parameter (`lst = []`) does **not** affect the caller; only in-place mutation does.

1.5.3 Positional, Keyword, and Mixed Calls

```
def move(src, dst, overwrite=False):
    ...

move("/tmp/a", "/tmp/b")                      # positional
move(src="/tmp/a", dst="/tmp/b", overwrite=True)  # keyword
move("/tmp/a", dst="/tmp/b")                  # mixed
```

Keyword arguments enhance readability and allow default values to be omitted selectively.

1.5.4 Default Argument Values

```
def stamp(message, timestamp=datetime.utcnow()):
    ...
```

*Defaults are evaluated **once**, at `def` time.*
Using a mutable object as a default can lead to shared state:

```
def collect(item, bucket=[]):       # bad
    bucket.append(item)
    return bucket
```

Idiomatic pattern:

```
def collect(item, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(item)
    return bucket
```

1.5.5 Variable-Length Parameters

Syntax	Purpose
*args	Pack extra **positional** arguments into a tuple.
**kwargs	Pack extra **keyword** arguments into a dict.

```
def log(msg, *values, **meta):
    for v in values:
        msg += f" {v}"
    if meta:
        msg += f" [{meta}]"
    print(msg)
```

You can also **unpack** when calling:

```
log("Coords:", *[10, 20], unit="km")
```

1.5.6 Positional-only and Keyword-only Parameters (3.8+)

```
def divmod_int(a, b, /):      # a, b are positional-only
    return a // b, a % b

def connect(*, host, port):   # host, port are keyword-only
    ...
```

Positional-only avoids accidental keyword use for performance-critical primitives; keyword-only clarifies call sites.

1.5.7 Annotations and Type Hints

```
from typing import Iterable, List
```

```python
def flatten(seq: Iterable[List[int]]) -> list[int]:
    return [item for sub in seq for item in sub]
```

Annotations **do not** enforce types at runtime; they are for linters, IDEs, and static checkers like `mypy`.
Retrieve them via `__annotations__` or the `inspect` module.

1.5.8 First-Class and Higher-Order Functions

```python
def twice(f, x):
    return f(f(x))

def increment(n):
    return n + 1

print(twice(increment, 3))   # 5
```

Because functions are values, you can build **callback systems**, **dispatch tables**, or pass behaviour into library code.

1.5.9 Lambda Expressions

A lambda is an anonymous, single-expression function.

```python
square = lambda n: n * n
```

Use only when the intent is obvious; prefer `def` for anything non-trivial to keep stack traces readable.

1.5.10 Closures

A closure remembers the environment in which it was created.

```
def make_counter():
    count = 0
    def inc():
        nonlocal count
        count += 1
        return count
    return inc

c = make_counter()
c(), c(), c()    # (1, 2, 3)
```

`nonlocal` allows the inner function to modify a variable in the **enclosing, non-global** scope.

1.5.11 Decorators

A decorator wraps one function with another.

```
from functools import wraps

def timing(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.3f}s")
```

```
        return result
    return wrapper

@timing
def heavy():
    ...
```

- `@wraps` copies metadata (`__name__` , `__doc__`) from the original function—important for introspection.
- Decorators can accept their own arguments by adding another layer of nesting.

1.5.12 Generator Functions

A `yield` turns a function into a **generator** that lazily produces a sequence.

```
def count_up(limit):
    n = 0
    while n < limit:
        yield n
        n += 1
```

State is preserved between `yield` *s*, making generators memory-efficient for large sequences.
`yield from iterable` delegates part of the generation to another iterator.

1.5.13 Recursion

Python supports recursion but has a default call-depth limit (`sys.getrecursionlimit()` , ~1000).
Tail-call optimisation is **not** automatic in CPython; iterative solutions are often preferred for deep loops.

1.5.14 Standard-Library Helpers

- `functools.lru_cache` – memoisation decorator.
- `functools.partial` – pre-fills some arguments.
- `itertools` – lazy functional pipelines (`map` , `filter` , `compress` , `islice` , …).
- `contextlib.contextmanager` – write a function that works with the `with` statement using `yield` .

1.5.15 Introspection

```
import inspect
def demo(a, b=2): ...
print(inspect.signature(demo))           # (a, b=2)
print(demo.__defaults__)                 # (2,)
print(demo.__annotations__)              # {}
```

Introspection enables dynamic dispatch, plug-in systems, and automatic CLI builders.

1.5.16 Performance Notes

- Local variables are faster than globals; default-value bindings exploit this:

```
def add(x, y, _add=int.__add__):   # caches lookup
    return _add(x, y)
```

- Avoid expensive default arguments (e.g., creating large lists) because they are built **once**.

1.5.17 Exercises

1. **Keyword-only Logger**
   Write `log(msg, *, level="INFO")` that prints `[LEVEL] msg`. Enforce keyword-only for `level`.
2. **LRU Fibonacci**
   Implement `fib(n)` with `@lru_cache(maxsize=None)` and time it against a non-cached version.
3. **Decorator with Arguments**
   Create `@retry(times=3)` that re-invokes the wrapped function if it raises `Exception`.
4. **Closure Counter**
   Build three independent counters from `make_counter()` and show that each maintains its own state.
5. **Generator Pipeline**
   Compose generator functions to read a file, strip whitespace, filter out blank lines, and yield line numbers plus text.
6. **Positional-only Division**
   Define `def div(a, b, /, precision=2)` that forbids keyword use of `a` and `b` but allows `precision=4`.

1.6 - Modules and Packages

A *module* is a single `.py` file that defines names—variables, functions, classes.
A *package* is a directory containing an optional `__init__.py`; it can hold sub-modules and sub-packages, forming a hierarchy.
Mastering the import system is essential for clean architecture, reuse, and eventual distribution of your code.

1.6.1 Importing a Module

```
import math
from math import sqrt, pi
import random as rnd
```

- `import module` binds the module object to a name.
- `from module import name` binds only the requested attributes.
- `as alias` creates a shorter or conflict-free reference.

1.6.2 Module Execution Context

Inside every module, two dunder variables are pre-defined:

```
print(__name__)     # 'my_module' or '__main__'
print(__package__)  # 'pkg' or '' (top level)
```

Use the **script guard** to make test code run only when the file is executed directly:

```
def _demo():
    ...

if __name__ == "__main__":
    _demo()
```

1.6.3 Module Search Path

```
import sys, pprint
pprint.pprint(sys.path)
```

Search order:

1. Directory of the running script (or CWD in interactive mode)
2. Zip-archives listed on `sys.path`
3. Standard-library directories
4. Entries in the `PYTHONPATH` environment variable

You can append to `sys.path` at runtime, but prefer **virtual environments** or proper packaging over manual path hacking.

1.6.4 Packages and Sub-packages

```
mypkg/
    __init__.py
    core.py
    utils/
        __init__.py
        helpers.py
```

```python
from mypkg.core import Engine
from .helpers import slugify          # relative import (inside utils)
```

- Absolute imports (`mypkg.core`) are explicit and recommended by **PEP 328**.
- Relative imports (starting with `.`) are allowed **only** inside packages.

Namespace Packages (PEP 420)

If `__init__.py` is omitted, Python treats the directory as a *namespace package* whose contents can be spread across multiple locations—useful for large plug-in ecosystems.

1.6.5 The `__init__.py` File

Goals:

- Establish sub-module exports via `__all__`
- Provide convenient *public* names re-exported at package level
- Run minimal, quick-to-import code (avoid heavy I/O)

```
# mypkg/__init__.py
from .core import Engine, Version       # re-export
__all__ = ["Engine", "Version"]
```

1.6.6 Reloading Modules (Development Convenience)

```
import importlib, mypkg.core
importlib.reload(mypkg.core)
```

Useful in REPL workflows; avoid in production code because it can leave objects in inconsistent states.

1.6.7 Virtual Environments

Create an isolated interpreter plus site-packages directory:

```
python3 -m venv .venv
source .venv/bin/activate   # Linux/macOS
.\.venv\Scripts\activate    # Windows
```

All `pip` installs now live under `.venv/`, preventing dependency conflicts.

1.6.8 Installing External Packages

```
pip install requests
pip freeze > requirements.txt
pip install -r requirements.txt
```

Use exact versions in production (`requests==2.32.3`) to ensure reproducible builds.

1.6.9 Distributing Your Own Package

Minimal `pyproject.toml` (PEP 621 syntax):

```
[project]
name = "mypkg"
version = "0.1.0"
description = "Sample package"
authors = [{ name = "You", email = "you@example.com" }]
requires-python = ">=3.9"
dependencies = ["requests>=2.31"]

[project.scripts]
mypkg = "mypkg.cli:main"
```

Build and upload:

```
python -m pip install build twine
python -m build                          # creates wheel and sdist
python -m twine upload dist/*
```

`console_scripts` entry points generate cross-platform command-line wrappers.

1.6.10 `__all__` and `import *`

```
__all__ = ["PublicClass", "public_fn"]
```

- Controls what `from module import *` brings into the caller's namespace.
- Having `__all__` makes intent explicit; otherwise * imports pull every name not starting with underscore.

1.6.11 Resource Files Inside Packages

Python 3.9+:

```
from importlib.resources import files
data = files("mypkg").joinpath("data/wordlist.txt").read_text()
```

This works even when the package is inside a zip or a wheel.

1.6.12 Best-Practice Checklist

Guideline	Reason
Prefer absolute imports	Clear, resilient to package moves
Keep **init**.py lightweight	Faster start-up, avoids circular imports
Encapsulate side effects in functions (main())	Imports should not run I/O
Use virtual environments	Dependency isolation
Pin versions in requirements.txt	Reproducibility
Provide a CLI via console_scripts	Easy user access
Document public API in **all**	Prevent accidental breakage

1.6.13 Exercises

1. **Package Skeleton**
   Create `calc/` with `__init__.py`, `add.py`, `sub.py`. Re-export `add` and `sub` functions at package level. Demonstrate usage from a parent directory.
2. **Import Cycle Detection**
   Build two modules `a.py` and `b.py` that import each other. Observe the runtime error and refactor to remove the cycle.
3. **Namespace Package Experiment**
   Create two separate directories, each containing `plugins/spam.py` and `plugins/eggs.py` (without `__init__.py`). Add both parent paths to `PYTHONPATH` and show that `import plugins.spam` finds both.
4. **Module Reload Pitfall**
   Write a module with a mutable global list. Import it, mutate the list from the main script, reload the module, and inspect how state is preserved.
5. **Console Script**
   Build a small package `echo` with `cli.py` exposing `main()`. Add a `console_scripts` entry point in `pyproject.toml`, build a wheel,

install it in a venv, and verify that the `echo` command runs.

1.7 - File Handling

Reading from and writing to files is a core task in almost every Python program.
This section explores the `open` interface, text-vs-binary modes, path manipulation, common data formats, and performance and safety considerations.

1.7.1 The `open` Built-in

```
fh = open("report.txt", mode="r", encoding="utf-8", newline="")
data = fh.read()
fh.close()
```

`open(file, mode="r", buffering=-1, encoding=None, errors=None, newline=None)`

Mode	Meaning
"r"	read (default)
"w"	write, truncating existing file
"a"	append (create if needed)
"x"	exclusive creation, fail if file exists
"+"	update (read + write)
Add "b"	binary mode ("rb", "wb", …)

Always close files to flush buffers and release OS handles. Prefer the context-manager form:

```
with open("log.txt", "a", encoding="utf-8") as log:
    log.write("Started\n")
```

Inside a `with` block, `__enter__` opens and `__exit__` closes, even on exceptions.

1.7.2 Text vs Binary

Text mode decodes bytes to `str` using the specified `encoding` (default platform encoding).
Binary mode (`"rb"` , `"wb"`) returns `bytes` unchanged.

```
with open("image.jpg", "rb") as img:
    raw = img.read(1024)
```

Choose binary when you handle non-text data (executables, packets, compressed files) or when precise byte counts matter.

1.7.3 Reading Patterns

Method	Description
fh.read()	Reads whole file into memory. Not suitable for very large files.
fh.readline()	Reads one line. Returned line retains trailing newline.
fh.readlines()	Reads all lines into a list.
Iteration	for line in fh: is lazy and memory-efficient.
fh.read(size)	Reads size bytes; returns shorter at EOF.

```python
def tail(path, n=10):
    with open(path, encoding="utf-8") as f:
        return f.readlines()[-n:]
```

1.7.4 Writing Patterns

Text: write `str`; *binary*: write `bytes`.

```python
with open("payload.bin", "wb") as out:
    out.write(b"\x90" * 16)
```

Call `out.flush()` or `out.close()` to guarantee data is on disk.
Buffered writes are faster; disable buffering with `buffering=0` only when necessary.

1.7.5 File Positioning

```python
f.seek(0, 2)      # move to end (whence=2)
size = f.tell()
f.seek(0)         # rewind
```

Modes:

- `0` start of file
- `1` current position
- `2` end of file

`seek()` on text files is limited to offsets returned by `tell()`; binary mode has full byte control.

1.7.6 The `pathlib` Module

`pathlib.Path` offers an object-oriented API replacing most `os.path` calls.

```
from pathlib import Path
p = Path("reports") / "2025" / "summary.csv"
if p.exists():
    print(p.stat().st_size, "bytes")
p.parent.mkdir(parents=True, exist_ok=True)   # create directories
p.write_text("header, value\n")
```

Important methods: `.iterdir()`, `.glob("*.py")`, `.read_bytes()`, `.write_bytes()`, `.resolve()`, `.rename()`, `.unlink()`.

1.7.7 Directory and Metadata Operations

```
import os, shutil, stat
os.listdir(".")
os.makedirs("backup", exist_ok=True)
shutil.copy2("source.txt", "backup/source.txt")   # preserves mtime
os.chmod("script.sh", stat.S_IRUSR | stat.S_IXUSR)
```

`shutil` provides high-level functions (`copytree`, `rmtree`, `make_archive`) that combine multiple system calls safely.

1.7.8 Temporary Files and Atomic Writes

```
from tempfile import NamedTemporaryFile
with NamedTemporaryFile("w", delete=False) as tmp:
    tmp.write("draft")
    temp_name = tmp.name
os.replace(temp_name, "final.txt")   # atomic swap
```

Writing to a temp file then renaming prevents partial writes being observed by other processes.

1.7.9 Memory-Mapped Files

For very large binary data or random read/write access:

```
import mmap
with open("disk.img", "r+b") as fh:
    mm = mmap.mmap(fh.fileno(), 0)
    print(mm[0x100:0x108])
    mm.close()
```

The file is mapped into virtual memory; slices return `bytes` without extra copying.

1.7.10 Common Structured Formats

JSON

```python
import json
with open("conf.json") as f:
    cfg = json.load(f)
cfg["enabled"] = True
with open("conf.json", "w") as f:
    json.dump(cfg, f, indent=2)
```

`json.dumps` / `json.loads` work with strings; `.dump` / `.load` with file handles.

CSV

```python
import csv
with open("hosts.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["ip"], row["hostname"])
```

Always pass `newline=""` to `open` when using `csv` to prevent subtle newline bugs on Windows.

Pickle (Caution)

`pickle` serialises arbitrary Python objects but executes arbitrary code on load.
Never unpickle data from an untrusted source.

Compressed Files

```python
import gzip, bz2
text = gzip.open("log.gz").read().decode()
```

Context-manager and file-like behaviour mirror `open`.

1.7.11 Exception Handling

```python
from pathlib import Path
try:
    data = Path("config.ini").read_text()
except FileNotFoundError:
    print("Missing config; using defaults.")
except PermissionError:
    print("Run with correct privileges.")
```

Catch specific subclasses of `OSError` (`IsADirectoryError`, `FileExistsError`, …) rather than the broad base class.

1.7.12 Performance Guidelines

- Read and write in **chunks** (`read(65536)`) for large files.
- Use binary mode to avoid per-character encoding overhead when copying raw data.
- Prefer `Path.read_bytes()` / `.write_bytes()` for concise code paths.
- Memory-map (`mmap`) for random access on multi-gigabyte files.
- Avoid many small writes; accumulate into a buffer or use `io.BufferedWriter`.

1.7.13 Security Checklist

Concern	Mitigation
Path traversal (../../etc/passwd)	Validate or normalise user-supplied paths with Path.resolve(), then check they stay inside an allowed directory.
Untrusted pickle data	Use JSON, MessagePack, or custom formats instead.

Concern	Mitigation
Race conditions (TOCTOU)	Open files with appropriate flags ("xb" for exclusive), or use temp-write-then-rename.
Encoding pitfalls	Always set encoding="utf-8" unless you have a specific reason not to.
Leaked file handles	Use with blocks everywhere.

1.7.14 Exercises

1. **Chunked Copier**
   Implement `copy(src, dst, chunk_size=262144)` that copies any size file using binary reads and writes, reporting bytes per second.
2. **Line Reverser**
   Read `input.txt` and create `output.txt` where line order is reversed but internal character order is preserved.
3. **Safe Config Writer**
   Write a function `save_json(path, obj)` that atomically writes pretty-printed JSON by using `NamedTemporaryFile` and `os.replace`.
4. **Directory Tree Size**
   Using `pathlib`, compute total size (in bytes) of all regular files under a directory, skipping symlinks.
5. **CSV Aggregator**
   Combine every `.csv` in a folder into a single `combined.csv` with an additional column `source_file` indicating origin.
6. **Memory-Mapped Patch**
   Open a large binary file with `mmap`, locate all occurrences of the byte sequence `b"\x90\x90\x90\x90"`, and overwrite them with `b"\xCC\xCC\xCC\xCC"`.

1.8 - Exception Handling

Errors are inevitable. Python's **exception mechanism** provides a structured, readable way to detect, propagate, and recover from abnormal situations without scattering manual error checks throughout your code.

1.8.1 The Exception Model

- When an error occurs, Python **raises** an *exception object*.
- If the current frame has no matching `except` clause, the exception propagates up the call stack.
- If it reaches the top level unanswered, the interpreter prints a **traceback** and halts.

```
>>> 1 / 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

An exception is therefore both **data** (the object) and **control flow** (the unwinding).

1.8.2 The `try / except / else / finally` Block

```
try:
    risky()
except ValueError as exc:
    handle(exc)
else:
    post_success()
finally:
    clean_up()
```

Clause	Executed when
try	Normal path.
except	A matching exception is raised. Stops propagation.

Clause	Executed when
else	try completed with *no* exception. Frequently used for code that must *not* run if errors occurred.
finally	Always runs, regardless of exceptions or return. Ideal for releasing resources.

Multiple `except` clauses are checked in order. Use parentheses to catch several exceptions at once:

```
except (FileNotFoundError, PermissionError) as exc:
    ...
```

Avoid the blanket `except Exception:` unless you immediately re-raise or log, because it hides programming mistakes.

1.8.3 Raising Exceptions

```
if bandwidth == 0:
    raise ZeroDivisionError("Bandwidth cannot be zero")

raise RuntimeError from original_exc   # explicit chaining
```

- `raise` with no argument inside an `except` re-raises the current exception.
- `X from Y` sets `__cause__`; the default implicit link is `__context__`.

1.8.4 Exception Hierarchy (abridged)

```
BaseException
 ├── SystemExit
```

```
  ├── KeyboardInterrupt
  ├── GeneratorExit
  └── Exception
       ├── ArithmeticError
       │    ├── ZeroDivisionError
       │    └── OverflowError
       ├── LookupError
       │    ├── IndexError
       │    └── KeyError
       └── ...
```

Catch `Exception` to include **most** runtime errors; catching `BaseException` will also swallow `SystemExit` and `KeyboardInterrupt`, which is rarely what you want.

1.8.5 Custom Exception Classes

Create domain-specific errors by subclassing `Exception` (or a narrower base). Keep them simple; often an empty class docstring is enough.

```python
class ConfigError(Exception):
    """Raised when configuration file is malformed."""
```

Custom attributes allow richer context:

```python
class HTTPError(Exception):
    def __init__(self, status, url):
        super().__init__(f"{status} for {url}")
        self.status = status
        self.url = url
```

1.8.6 `else` and `finally` in Practice

```
def load_and_parse(path):
    try:
        data = Path(path).read_text(encoding="utf-8")
    except FileNotFoundError:
        log.error("Missing file %s", path)
        raise
    else:
        return json.loads(data)      # only runs if open succeeded
    finally:
        log.debug("load_and_parse finished")  # always executes
```

`else` avoids accidentally wrapping code that might itself raise and be wrongly attributed to the file operation.

1.8.7 The EAFP Principle

Easier to Ask Forgiveness than Permission drives idiomatic Python:

```
try:
    cache[item]
except KeyError:
    cache[item] = compute(item)
```

Attempt the operation and handle the specific failure, rather than pre-checking with `if`.

1.8.8 Logging vs Swallowing

Silent failure hides bugs. At minimum, **log** unexpected exceptions before discarding them:

```
import logging
log = logging.getLogger(__name__)

try:
    work()
except Exception:
    log.exception("Unexpected error")   # writes traceback
    raise                                # or decide to continue
```

1.8.9 Chaining and Re-raising

```
try:
    connect()
except TimeoutError as exc:
    raise ConnectionError("Upstream timeout") from exc
```

Explicit chaining (`from`) keeps the original stack trace while translating to an API-level error.

1.8.10 Context Managers for Safe Cleanup

`with` relies on `__enter__` / `__exit__` . The latter gets any exception details:

```python
class suppress:
    def __init__(self, *types):
        self.types = types
    def __enter__(self): ...
    def __exit__(self, exc_type, exc, tb):
        return exc_type is not None and issubclass(exc_type, self.types)

with suppress(FileNotFoundError):
    Path("optional.txt").unlink()
```

Prefer existing managers: `pathlib.Path.open`, `tempfile`, `contextlib` (`suppress`, `closing`, `ExitStack`, etc.).

1.8.11 New in 3.11: `ExceptionGroup` and `except*`

Asynchronous and parallel code can raise many errors at once. `ExceptionGroup` encapsulates them; `except*` handles sub-groups selectively.

```python
try:
    await gather(*tasks)
except* ValueError as eg:
    ...
```

If targeting ≤3.10, wrap tasks individually.

1.8.12 Best-Practice Checklist

Guideline	Why
Capture the **narrowest** exception type possible.	Prevents masking unrelated bugs.
Prefer EAFP (try/except) over LBYL (if/else) for I/O and look-ups.	Race-free and concise.
Use else for code that must not run on error.	Keeps scope clear.
Always clean up in finally or via context managers.	Avoids resource leaks.
Never blanket-catch BaseException; rarely blanket-catch Exception.	Allows Ctrl-C / system exits to propagate.
Log unexpected exceptions before suppressing.	Aids debugging.
Keep custom exception classes minimal.	Maintainable API.

1.8.13 Exercises

1. **Safe Divider**
   Write `safe_div(a, b, default=None)` that returns `a/b` but *logs* and returns `default` if division fails for any arithmetic reason (zero or type error), without hiding other exceptions.
2. **Atomic Writer Context Manager**
   Implement `AtomicWrite(path)` as a context manager that yields a temporary writable file handle and, on successful exit, replaces the target file; on exception, deletes the temp file.
3. **Selective Retry Decorator**
   Create `@retry(exc_types, times=3)` that re-calls the function when it raises one of `exc_types`, but re-raises immediately for all other exceptions.
4. **Exception Group Splitter (>=3.11)**
   Launch three coroutines that intentionally raise different exceptions. Catch the resulting `ExceptionGroup` and handle each exception type separately with `except*`.
5. **Traceback Formatter**

Use the `traceback` module to write a function `format_exception(e)` that returns a coloured, one-line traceback string for logging.

1.9 - Writing “Pythonic” Code and Style Guidelines

“Pythonic” means more than just following syntax rules.
It is a set of **cultural** conventions—codified in *PEP 8*, *PEP 20* (the Zen of Python), and community practice—that make code **readable, predictable, and idiomatic**.

1.9.1 The Zen of Python (PEP 20)

```
import this
```

Key take-aways:

Principle	Practical Implication
Readability counts	Choose clarity over micro-optimisations.
There should be one— and preferably only one—obvious way to do it	Prefer idioms that other Python developers recognise.
Simple is better than complex	Break big functions into smaller ones; avoid deep nesting.
Namespaces are one honking great idea	Use modules and packages to prevent name collisions.

1.9.2 PEP 8 Basics

Topic	Guideline
Indentation	4 spaces, never tabs.

Topic	Guideline
Line length	79 chars for code, 72 for docstrings (tools like *black* default to 88).
Blank lines	Two blank lines between top-level defs; one inside classes.
Imports	Three blocks: standard lib, third-party, local; each alphabetised.
Naming	snake_case for variables & functions, PascalCase for classes, UPPER_SNAKE for constants.
Whitespace	spam(ham[1], {eggs: 2}), not spam (ham[1] , { eggs : 2 }).
Comparisons to None	Use is None / is not None.
Boolean operators	Prefer if not seq: to if len(seq) == 0:.

Automate enforcement:

```
pip install black isort flake8
black src/
isort src/
flake8 src/
```

1.9.3 Docstrings (PEP 257)

```
def add(a: int, b: int) -> int:
    """Return the sum of *a* and *b*.

    Parameters
    ----------
    a, b : int
        Operands to add.
```

```
    Returns
    -------
    int
        The sum.
    """
```

- One-line summary ends in a period.
- Blank line, then detailed description or parameter list.
- Use reStructuredText, Google, or NumPy style—stay consistent.

1.9.4 Type Hints (PEP 484, PEP 563, PEP 644)

- Type hints **do not** affect runtime by default.
- Run static checkers: `mypy`, `pyright`, `pylance`.

```python
from typing import Iterable, Any

def first(seq: Iterable[Any]) -> Any | None:
    for item in seq:
        return item
    return None
```

Prefer `list[str]` over `List[str]` on Python 3.9+.

1.9.5 Idiomatic Patterns

Task	Idiomatic	Non-idiomatic
Loop with index	for i, x in enumerate(seq):	for i in range(len(seq)):
Default dict entry	cache.setdefault(key, []).append(val)	if key not in cache: …
Combine strings	",".join(parts)	s = ""; for p in parts: s += p
Conditional assignment	status = "open" if ok else "closed"	if ok: status = "open" …
File I/O	with open(path) as f:	f = open(path); …; f.close()

1.9.6 EAFP vs LBYL

Easier to Ask Forgiveness than Permission:

```
try:
    result = cache[key]
except KeyError:
    result = cache[key] = compute(key)
```

This avoids race conditions and extra look-ups.

1.9.7 Comprehensions and Generators

- List/Dict/Set comprehensions for simple transforms.
- Switch to a generator expression to avoid building an intermediate list when iterating once.

```
total = sum(int(line) for line in file)
```

Nested comprehensions quickly become unreadable—extract helper functions.

1.9.8 Context Managers

Use `with` to manage **any** resource that needs teardown (files, locks, database cursors). Create your own via:

```
from contextlib import contextmanager

@contextmanager
def change_dir(path: Path):
    prev = Path.cwd()
    os.chdir(path)
    try:
        yield
    finally:
        os.chdir(prev)
```

1.9.9 Data Classes vs NamedTuples vs Regular Classes

Feature	dataclass	namedtuple	Manual class
Boilerplate	Low	Very low	High
Mutability	Mutable (frozen optional)	Immutable	Depends
Default methods	**init**, **repr**, **eq**	Same	Manual
Type hints	Yes	Yes (positional)	Yes

Prefer `@dataclass(frozen=True)` for lightweight value objects.

1.9.10 Common “Code Smells”

Smell	Remedy
Long function (>50 lines)	Refactor into helpers or a class.
Deep nesting (>3 levels)	Early return or guard clauses.
Duplicate code blocks	Extract shared function / mixin.
Magic numbers	Replace with named constants (MAX_RETRIES = 5).
Broad except:	Catch specific exceptions; log and re-raise others.

Automated detectors: `pylint`, `radon`, `flake8-bandit`.

1.9.11 Import Time and Side Effects

- Modules should be **import-safe**: importing shouldn’t start network calls or heavy computation.
- Put script entry points behind `if __name__ == "__main__":`.

1.9.12 Performance Without Premature Optimisation

- Profile first: `python -m cProfile -o stats.prof your_app.py`.
- Tight loops: move look-ups to locals; consider `itertools`, `functools`.
- For CPU-bound tasks, look to *Cython*, *Numba*, or multiprocessing.

- For I/O-bound tasks, prefer `asyncio` or threading; use non-blocking libraries (`aiohttp` , `aiomysql`).

1.9.13 Testing and Continuous Integration

- Write small, deterministic tests with `pytest` .
- Measure coverage: `pytest --cov=src` .
- Lint, test, and type-check in CI (GitHub Actions, GitLab CI, etc.).

Sample GitHub Actions fragment:

```
- uses: actions/setup-python@v5
  with: {python-version: "3.12"}
- run: pip install -r requirements-dev.txt
- run: black --check src
- run: isort --check src
- run: mypy src
- run: pytest -q
```

1.9.14 Exercises

1. **PEP 8 Refactor** – The snippet below violates several style rules. Re-write it Pythonically.

```
def compute(a,b):
  if a>0 and b>0: return a+b
  else:
     return(a-b)
```

2. **Docstring Practice** – Add a NumPy-style docstring (parameters, returns, examples) to `compute`.
3. **Type-Check** – Annotate `compute` and run `mypy` so it passes with `strict=True`.
4. **Context Manager** – Implement a `Timer()` context manager that prints elapsed time on exit and use it to wrap a call to `sum(range(10**7))`.
5. **Data Class Equality** – Create a frozen dataclass `Point(x: int, y: int)` and demonstrate that two points with the same coordinates compare equal and are hashable.

1.10 - Object-Oriented Programming (OOP)

Python's object model is simple but remarkably powerful: **everything is an object**, even modules, functions, and classes themselves. Understanding how to design, extend, and introspect classes will unlock a large part of the standard library and make your own code clearer and more reusable.

1.10.1 Creating Classes

```
class Point:
    """Immutable 2-D point."""
    __slots__ = ("x", "y")      # optional memory optimisation

    def __init__(self, x: float, y: float) -> None:
        self.x = x
        self.y = y

    def distance_to(self, other: "Point") -> float:
        return ((self.x - other.x) ** 2 + (self.y - other.y) ** 2) ** 0.5

    def __repr__(self) -> str:             # developer-facing string
        return f"Point({self.x}, {self.y})"
```

- `self` is just a convention—any first parameter name works, but never change it.
- `__slots__` prevents the creation of `__dict__`, saving memory for millions of small objects.
- `__repr__` should, when feasible, yield valid Python code that reconstructs the object.

1.10.2 Attributes, Properties, and Encapsulation

Python trusts the programmer—private variables are a convention (`_internal`) rather than enforced.
Use **properties** when field access needs validation or lazy computation:

```
class Temperature:
    def __init__(self, celsius: float) -> None:
        self._c = celsius

    @property
    def celsius(self) -> float:
        return self._c

    @celsius.setter
    def celsius(self, value: float) -> None:
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._c = value

    @property
    def fahrenheit(self) -> float:
        return self._c * 9 / 5 + 32
```

1.10.3 Class Variables, Class Methods, Static Methods

```python
class Session:
    _active: int = 0                        # class variable

    def __init__(self) -> None:
        Session._active += 1

    def close(self) -> None:
        Session._active -= 1

    @classmethod
    def active(cls) -> int:               # receives the *class*
        return cls._active

    @staticmethod
    def banner() -> str:                  # receives nothing implicitly
        return "Secure Session"
```

Class variables are shared across all instances; instance attributes live in `obj.__dict__`.

1.10.4 Inheritance and the Method Resolution Order (MRO)

```python
class Animal:
    def speak(self): ...

class Canine(Animal):
    def speak(self): print("Bark!")

class WolfMixin:
    def howl(self): print("Awooo!")
```

```
class Dog(Canine, WolfMixin):            # multiple inheritance
    pass
```

Python uses **C3 linearisation** to compute the MRO (`Dog.__mro__`).
If two parents implement the same method, the left-most base class wins.

Prefer **composition** over deep inheritance trees:

```
class Engine: ...
class Car:
    def __init__(self):
        self.engine = Engine()           # Car *has-a* Engine
```

1.10.5 Abstract Base Classes (ABCs) and *Protocols*

```
from abc import ABC, abstractmethod

class Downloader(ABC):
    @abstractmethod
    def fetch(self, url: str) -> bytes: ...

class HTTPDownloader(Downloader):
    def fetch(self, url): ...
```

Abstract methods enforce an interface at **instantiation** time.

Since Python 3.8, **static typing** may use *protocols* (structural subtyping):

```python
from typing import Protocol

class Runnable(Protocol):
    def run(self) -> None: ...

def start(task: Runnable): task.run()
```

Any object with a `run()` method satisfies `Runnable`, no inheritance required.

1.10.6 Special (Dunder) Methods

Purpose	Method(s)
String conversion	**str**, **repr**
Arithmetic	**add**, **sub**, …
Comparisons	**eq**, **lt**, …
Container behaviour	**len**, **getitem**, **iter**
Context manager	**enter**, **exit**
Callable objects	**call**

Example: making `Point` add like vectors:

```python
def __add__(self, other: "Point") -> "Point":
    return Point(self.x + other.x, self.y + other.y)
```

1.10.7 Data Classes and `attrs`

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class User:
    id: int
    name: str
```

`@dataclass` auto-generates `__init__`, `__repr__`, `__eq__`, and (if `frozen=True`) `__hash__`.
Third-party libraries like **attrs** and **pydantic** provide richer validation and serializers.

1.10.8 Metaclasses (Advanced)

A *metaclass* customises class creation:

```
class Singleton(type):
    _cache = {}
    def __call__(cls, *a, **kw):
        if cls not in cls._cache:
            cls._cache[cls] = super().__call__(*a, **kw)
        return cls._cache[cls]

class Config(metaclass=Singleton):
    pass
```

Metaclasses power ORMs, testing frameworks, and `abc.ABCMeta`, but should be a *last resort*—prefer class decorators or mixins first.

1.10.9 Performance Considerations

- Attribute lookup (`obj.attr`) is a hash-table search; avoid it inside tight C-level loops—cache to locals.
- `__slots__` removes `__dict__` , shrinking memory and speeding lookup, but blocks new attributes and multiple-inheritance unless every base defines slots.
- `weakref.WeakValueDictionary` lets you cache instances without preventing garbage collection.

1.10.10 Design Principles

Principle	Application
Single Responsibility	A class models one concept (e.g., Parser, Lexer).
Composition > Inheritance	Fewer cross-class couplings; easier testing.
Open/Closed	Subclass or inject behaviour instead of modifying core code.
Liskov Substitution	Subclasses must honour base-class contracts.
Explicit is better than implicit	Favour readable code over magic via metaclasses.

1.10.11 Exercises

1. **Vector Class**
   Implement `Vector2D` with `__add__` , `__sub__` , `__abs__` , and `__iter__` . Support hashing when frozen.
2. **Property Validation**
   Create `Account(balance)` where `balance` never drops below 0; deposit/withdraw methods must use the property.
3. **ABC Plugin System**
   Design an `abstract class Command` with `name` and `run(args)` ; dynamically load subclasses from a `plugins/` directory and

execute them.

4. **Slots vs Dict Benchmark**
   Measure memory (`sys.getsizeof`) of 1 000 000 plain objects vs slotted objects holding two floats.
5. **Protocol Compliance**
   Write a function `serialize(obj)` that accepts anything meeting the protocol `{ to_json() -> str }`. Show it working with a data class that defines `to_json`.